

P2905R2

Runtime format strings

Published Proposal, 2023-07-23

Author:

[Victor Zverovich](#)

Audience:

LWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

- 1 Introduction**
- 2 Problems**
- 3 Changes since R1**
- 4 Changes since R0**
- 5 Polls**
- 6 Proposal**
- 7 Impact on existing code**
- 8 Wording**
- 9 Implementation**

References

Informative References

§ 1. Introduction

[P2216] "std::format improvements" introduced compile-time format string checks which, quoting Barry Revzin, "is a fantastic feature" ([P2757]). However, due to resource constraints it didn't provide a good API for using formatting functions with format strings not known at compile time. As a workaround one could use type-erased API which has never been designed for that. This severely undermined safety and led to poor user experience. This paper fixes the safety issue and its companion paper [P2918] proposes direct support for runtime format strings which has been long available in the {fmt} library.

§ 2. Problems

[P2216] "std::format improvements" introduced compile-time format string checks for `std::format`. This obviously requires format strings be known at compile time. However, there are some use cases where format strings are only known at runtime, e.g. when translated through gettext ([GETTEXT]). One possible workaround is using type-erased formatting functions such as `std::vformat`:

```
std::string str = translate("The answer is {}.");
std::string msg = std::vformat(str, std::make_format_args(42));
```

This is not a great user experience because the type-erased API was designed to avoid template bloat and should only be used by formatting function writers and not by end users.

Such misuse of the API also introduces major safety issues illustrated in the following example:

```
std::string str = "{}";
std::filesystem::path path = "path/etic/experience";
auto args = std::make_format_args(path.string());
std::string msg = std::vformat(str, args);
```

This innocent-looking code exhibits undefined behavior because format arguments store a reference to a temporary which is destroyed before use. This has been discovered and fixed in [FMT] which now rejects such code at compile time.

§ 3. Changes since R1

- Added LEWG poll results for R1.

§ 4. Changes since R0

- Moved the API for direct runtime format string support to a separate paper [\[P2918\]](#) per LEWG feedback.
- Removed forwarding in `print` per LEWG feedback.

§ 5. Polls

LEWG poll results for R0:

POLL: In P2905R0 split `runtime_format` and `make_format_args` (add fix for `print`) into separate papers, the latter being a DR against C++23.

SF	F	N	A	SA
5	10	4	0	0

Outcome: Strong Consensus in Favor

LEWG poll results for R1:

POLL: Send P2905R1 (Runtime Format Strings) to electronic balloting to be forwarded to library for C++26, with the intention to retroactively apply the paper to C++23.

SF	F	N	A	SA
4	7	0	0	0

Outcome: Unanimous consent in favor.

§ 6. Proposal

This paper proposes changing `make_format_args` to take lvalue references instead of forwarding references, rejecting problematic code:

```
std::filesystem::path path = "path/etic/experience";  
auto args = std::make_format_args(path.string()); // ill-formed
```

This has been implemented in `{fmt}` catching some bugs even though the pattern of using `make_format_args` has never been suggested as a way to pass runtime format strings there. If left unchanged this will be a major safety hole in the standard formatting facility.

In the standard itself `make_format_args` is already called with lvalue references in `format`, e.g. [\[format.functions\]](#):

```
template<class... Args>  
    string format(format_string<Args...> fmt, Args&&... args);
```

Effects: Equivalent to:

```
return vformat(fmt.str, make_format_args(args...));
```

Notice that there is intentionally no forwarding of `args` so the switch from forwarding to lvalue references is effectively a noop there.

There is forwarding in the definitions on recently added `print` functions (e.g. [\[print.fun\]](#)) which is unnecessary and inconsistent with `format`. Removing forwarding there is not observable.

§ 7. Impact on existing code

Rejecting temporaries in `make_format_args` is an (intentionally) breaking change.

Searching GitHub for calls of `std::make_format_args` using the following query

```
"std::make_format_args" language:c++ -path:libstdc -path:libcxx -path:inclu
```

returned only 844 results at the time of writing. For comparison, similar search returned 165k results for `fmt::format` and 7.3k for `std::format`. Such low usage is not very surprising because `std::format` is not widely available yet.

At least 452 of these call sites use `make_format_args` as intended and will require no changes:

```
std::vformat_to(std::back_inserter(c), fmt.get(), std::make_format_args(arg
```

72 of remaining calls can be trivially fixed by removing unnecessary forwarding.

This leaves only 320 cases most of which will continue to work and the ones that pass temporaries can be easily fixed by either switching to `std::runtime_format` or by storing a temporary in a variable.

§ 8. Wording

Change in [\[format.syn\]](#):

```
namespace std {  
    ...  
  
    template<class Context = format_context, class... Args>  
        format_arg_store<Context, Args...>  
            make_format_args(Args&... fmt_args);  
    template<class... Args>  
        format_arg_store<wformat_context, Args...>  
            make_wformat_args(Args&... args);  
  
    ...  
}
```

Change in [\[format.arg.store\]](#):

```
template<class Context = format_context, class... Args>  
    format_arg_store<Context, Args...> make_format_args(Args&... fmt_args);
```

2 *Preconditions*: The type `typename Context::template formatter_type<remove_cvref const_t<T>_i>` meets the *BasicFormatter* requirements ([`formatter.requirements`]) for each `Ti` in `Args`.

...

```
template<class... Args>
    format_arg_store<wformat_context, Args...> make_wformat_args(Args&&... args);
```

Change in [`print.fun`]:

```
template<class... Args>
    void print(FILE* stream, format_string<Args...> fmt, Args&&... args);
```

2 *Effects*: If the ordinary literal encoding ([`lex.charset`]) is UTF-8, equivalent to:

```
vprint_unicode(stream, fmt.str, make_format_args(std::forward<Args>(args))...args);
```

Otherwise, equivalent to:

```
vprint_nonunicode(stream, fmt.str, make_format_args(std::forward<Args>(args))...args);
```

Change in [`ostream.formatted.print`]:

```
template<class... Args>
    void print(ostream& os, format_string<Args...> fmt, Args&&... args);
```

1 *Effects*: If the ordinary literal encoding ([`lex.charset`]) is UTF-8, equivalent to:

```
vprint_unicode(os, fmt.str, make_format_args(std::forward<Args>(args))...args);
```

Otherwise, equivalent to:

```
vprint_nonunicode(os, fmt.str, make_format_args(std::forward<Args>(args))...
```

§ 9. Implementation

The proposed API has been implemented in the {fmt} library ([FMT]).

§ References

§ Informative References

[FMT]

Victor Zverovich; et al. *The fmt library*. URL: <https://github.com/fmtlib/fmt>

[GETTEXT]

Free Software Foundation. *gettext*. URL: <https://www.gnu.org/software/gettext/>

[P2216]

Victor Zverovich. *std::format improvements*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p2216r3.html>

[P2757]

Barry Revzin. *Type-checking format args*. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2757r1.html>

[P2918]

Victor Zverovich. *Runtime format strings II*. URL: <https://isocpp.org/files/papers/P2918R0.html>